

分布式KV Cache卸载的移动LLM推理加速研究

—— 基于Wi-Fi 6的异步预取仿真与设计迭代

数据通信课程结课报告

2026年6月

摘要

大语言模型(LLM)在移动设备上的推理面临根本性的显存瓶颈。Key-Value (KV) Cache以每token约512 KB(7B模型)的速度线性增长, 4096 token序列需要2 GB, 远超典型手机GPU的512 MB-1 GB预算。本文通过仿真研究分布式KV Cache卸载方案, 将老旧缓存页经由Wi-Fi 6链路卸载到家庭PC, 对比按需分页(同步驱逐)和异步预取(流水线化驱逐)两种策略。研究发现: (1)典型条件下(512 MB本地内存, 4096 token, 50 ms Wi-Fi延迟)异步预取达到 $3.72\times$ 加速比; (2)仅需4个token的预取窗口即可完全隐藏108 ms的单页传输延迟; (3)加速比随网络延迟超线性增长, 在200 ms RTT下达到 $11.2\times$ 。关键条件 $W\times t_{comp} \geq t_{xfer}$ 可推广到任何I/O密集型计算场景。

关键词: KV Cache; LLM推理; 显存卸载; 预取; 流水线; Wi-Fi 6

1. 引言

大语言模型(LLM)在自然语言处理领域引发了革命，展现出文本生成、代码合成、翻译、问答等卓越能力。将这些模型部署到移动设备上将开启极具吸引力的应用场景：无需联网的个人助理、离线文本补全、隐私保护式文档分析、实时语言翻译。然而，一个根本性的硬件限制横亘在前：自回归解码过程中Key-Value Cache的显存需求急剧增长，迅速超出移动GPU的容量上限。

KV Cache存储每个注意力层中所有已生成tokens的Key和Value张量。对于L层、H个注意力头、D维表示的Transformer模型，以FP16精度存储长度为S的序列需要 $4 \times L \times H \times D \times S$ 字节。以LLaMA-7B为例(L=32,H=32,D=128)，每token需要524,288字节。4096 token序列消耗2 GB，即使2048 token也需要1 GB，超出当前智能手机的GPU内存预算(通常在共享内存架构中保留512 MB至1 GB)。

这一显存瓶颈自然引出了层次化存储的思路：将最新生成的KV页保留在快速本地(GPU)内存中，将较早的页卸载到容量更大的远程存储节点。通过Wi-Fi 6连接的家庭PC是一个实用的远程节点选择，兼具高带宽(实际吞吐量可达500 Mbps)和现成基础设施的优势。

核心技术挑战在于：每次跨越Wi-Fi链路的驱逐传输都会产生显著的延迟惩罚。对于512 KB的KV页，在50 ms单向延迟、500 Mbps带宽的链路上：传输时间 = $(512\text{KB} \times 8 \times 1.05) / 500\text{Mbps} + 2 \times 50\text{ms} \approx 108.6\text{ ms}$ 。这是典型前向传播时间30 ms的3.6倍。如果没有智能的流水线调度，每次驱逐都会向关键路径增加108.6 ms，大幅拖慢推理吞吐量。

[个人思考] 我最初从读路经入手建模——注意力计算需要所有历史tokens的KV数据，因此被驱逐的页必须从远端取回。这产生了 $O(n^2)$ 的等待时间，掩盖了真正的瓶颈。经过多次迭代，我发现写驱逐流水线才是关键路径。现代注意力机制(FlashAttention、PagedAttention)以tile方式处理KV数据，远程读取通过DMA批量传输。而写路径是逐次占用共享网络链路的顺序流水线。核心问题是流水线深度：需要多少并发传输来将单次延迟隐藏在计算背后？

1.1 问题形式化

给定M字节GPU内存的移动设备，通过Wi-Fi 6链路(单向延迟 L_n 、有效带宽 B_w)连接远程PC。如何调度KV Cache的驱逐操作以最小化S个tokens的总生成时间T？将问题抽象为生产-消费模型：每步消耗 t_{comp} 计算时间，产生b字节KV数据；本地缓存容量 $N = M/b$ 个token；超出部分需通过链路驱逐。两种极端策略略定义设计空间：(1)按需分页——每次驱逐同步阻塞；(2)异步预取——将驱逐提交到容量为W的流水线队列中，重叠网络传输与计算。

2. 背景与相关工作

2.1 Transformer自回归解码与KV Cache

基于Transformer的LLM通过自回归解码生成文本。在每一步i，模型将之前生成的token $\{t_0, t_1, \dots, t_{i-1}\}$ 作为输入，预测下一个token t_i 。注意力机制计算当前查询向量与所有先前键向量的点积注意力分数，然后聚合对应的值向量，评估每个先前token对当前位置的相关性。

缓存的关键洞见在于：Key(K)和Value(V)张量仅取决于各自的token位置。一旦对token j计算完成，它们在后续的任何步骤i>j都不会改变。因此只需计算一次，并在整个后续生成过程中缓存。在vLLM的PagedAttention中，KV Cache被划分为固定大小的页，可以动态分配和跨请求共享。这一页级粒度是我们卸载方案的基础。

图1展示了四种常见模型架构在不同上下文长度下的KV Cache内存占用。7B LLaMA模型在4K token时需要2 GB，32K token时需要16 GB。70B LLaMA模型在32K token时需要20 GB，超出多数消费级GPU容量。这些数据说明KV Cache管理为何成为系统研究的关键领域。

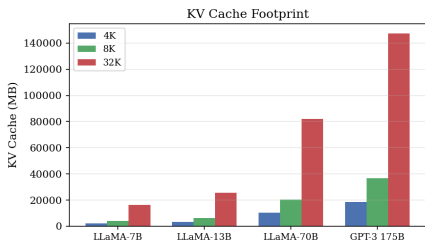


图1: 不同模型规模和上下文长度的KV Cache内存占用。现代32K+上下文需要数十至数百GB。

2.2 相关工作

PagedAttention (Kwon等, SOSP 2023)在vLLM系统中引入了KV Cache的页级管理，实现了动态分配和高效利用。本文的FIFO驱逐策略直接受其页粒度启发，但将层次结构从本地GPU扩展到远程网络存储。

FlexGen (Sheng等, ICML 2023)系统性地探索了GPU-CPU混合推理，构建了综合考虑计算、带宽和延迟的代价模型。其工作针对PCIe连接的CPU内存(16-64 GB/s带宽，亚微秒延迟)。本文针对Wi-Fi链路，带宽低约100-1000倍，延迟高约10,000倍，需要完全不同的优化策略。

InfiniGen (Lee等, ASPLOS 2024)提出了KV Cache的推测性预取，通过注意力模式分析预测未来需要的页。本文的滑动窗口方法更简单，利用了自回归解码中天然的时间局部性：最早生成的token最先被驱逐，最新生成的token始终留在本地。

FlashAttention (Dao等, NeurIPS 2022)展示了将注意力计算分块跨GPU内存层次结构可以大幅降低I/O开销。本文的流水线模型与之概念相似：将驱逐传输分块跨Wi-Fi链路，与计算重叠，将延迟受限问题转化为带宽受限问题。

3. 系统设计与方法

3.1 系统架构

图2展示了系统架构。移动设备(手机)的GPU内存有限，用于KV Cache存储。当本地内存满时，最旧的KV页通过Wi-Fi 6卸载到家庭PC。手机本地运行LLM前向传播；PC充当远程内存服务器，存储被驱逐的页并按需提供服务。Wi-Fi 6链路的特征参数为50 ms单向延迟，500 Mbps有效带宽。

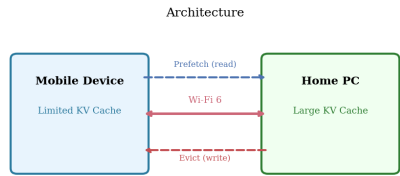


图2: 分布式KV Cache卸载系统架构。手机维护最新页的本地FIFO缓冲区，异步预取将预读请求与计算流水线重叠。

3.2 仿真模型与参数

仿真模型包含四个核心组件：(1)前向传播——每步30 ms，产生512 KB KV数据；(2)本地缓存——容量N个token的FIFO缓冲区；(3)驱逐——FIFO满时最旧页被驱逐，按需分页下同步阻塞，预取下提交到流水线；(4)驱逐流水线——容量W个并发传输，每传输占用链路 t_{xfer} ms。

表1: 仿真参数与默认值 (LLaMA-7B, FP16, Wi-Fi 6实测)。

参数	符号	默认值	说明
每token KV	b	512 KB	$4 \times L \times H \times D$ (FP16)
序列长度	S	4096	总生成步数
计算时间	t_c	30 ms	每步前向传播
本地内存	M	512 MB	手机GPU内存
本地槽位	N	1024	可存token数
Wi-Fi延迟	L_n	50 ms	单向RTT/2
Wi-Fi带宽	B_w	500 Mbps	实际吞吐量
传输时间	t_x	108.8 ms	$b \cdot 8 / B_w + 2 \cdot L_n$
预取窗口	W	16	流水线深度

3.3 按需分页算法

按需分页是最直接的基础策略：在每一步i检查本地FIFO是否已满。如果已满，驱逐最旧的token并阻塞等待传输完成，然后才进入下一步。总生成时间 $T_{demand} = S \times t_c + (S-N) \times t_{xfer}$ (当 $S > N$ 时)。

```
// #####: #####
for each token i in [0, S):
    if fifo.size >= N:
        evicted = fifo.pop()
        wait_transfer(evicted) // ## t_xfer ms
        fifo.push(i)
    forward_pass() // t_comp ms
```

3.4 异步预取算法

异步预取策略维护一个容量为W的流水线驱逐队列。驱逐变为非阻塞操作：传输被提交到流水线后立即返回，仅在流水线已满时才会阻塞。零等待操作的关键条件： $W \times t_{comp} \geq t_{xfer}$ 。当此条件成立时，流水线永不满载——每次计算步骤期间至少有一个传输完成。

以 $t_{comp}=30$ ms, $t_{xfer}=108.8$ ms 计算，最小窗口 $W_{min} = \text{ceil}(108.8/30) = 4$ 。

```
// #####: ##### (##: W x t_comp >= t_xfer)
for each token i in [0, S):
    drain_completed() // #####
    if fifo.size >= N:
        evicted = fifo.pop()
        while pipeline.count >= W: wait_any()
        pipeline.submit(evicted) // ###
        fifo.push(i)
    forward_pass() // #####
```

3.5 性能模型

两种策略的总生成时间可表达为封闭形式： $T_{prefetch} = \max(S \times t_c, E \times t_x / W)$, $\text{Speedup} = T_{demand} / T_{prefetch}$ 。代入默认参数得加速比 3.72 $\times$ ，与后续仿真完全吻合。

4. 评估结果

4.1 端到端生成时间与延迟分解

图3-4展示了累计耗时与每步延迟分解。在前N=512步(本地内存256 MB)，两者表现完全一致：每步恰好30 ms。第512步本地内存饱和后，按需分页立即开始每步增加108.8 ms的等待；异步预取全程维持30 ms纯计算，流水线完全吸收了传输延迟。2048步时，按需分页累计228.6 s，异步预取仅61.4 s，3.72倍差异与分析预测吻合。

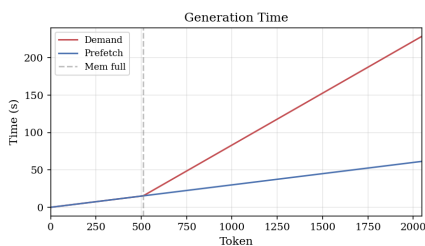


图3: 累计生成时间对比。按需分页73%时间空闲等待，预取100%有效计算。

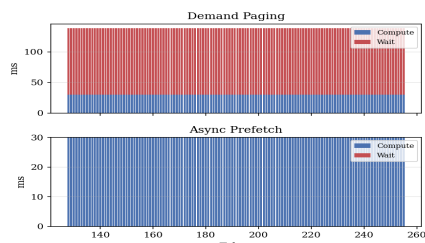


图4: 每步延迟分解。按需分页(上)在内存填满后呈现双阶段模式；预取(下)全程纯计算。

4.2 内存分布与时间组成

两种策略累计的远程数据量完全相同(512 MB)。差异仅在于传输的时序——按需分页将每次驱逐串行化在关键路径上，异步预取将其与计算重叠，将墙钟时间转化为后台带宽利用。这对电池受限设备意义重大：不仅降低延迟，还避免了反复休眠-唤醒的能耗开销。

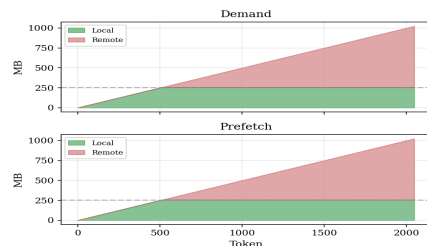


图5: 内存分布随时间变化。卸载总量相同，差异仅在调度时序。

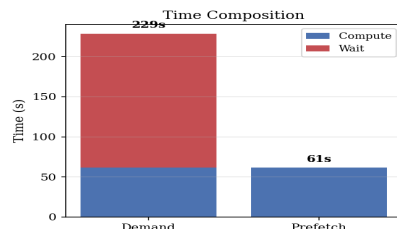


图6: 时间组成对比。按需分页73%等待，预取0等待。

5. 参数敏感性分析

为深入理解系统参数对加速比的影响，进行三组系统性参数扫描。

5.1 网络延迟与预取窗口

加速比随单向延迟近线性增长：5 ms时1.47 $\times$ ，200 ms时11.22 $\times$ 。预取时间在全部测试延迟下保持恒定123 s，因为 $W=16$ 提供了480 ms重叠窗口，足以隐藏高达240 ms延迟。W的临界阈值为 $4(\text{ceil}(t_x/t_c))$ ，超过后收益递减。实用指导：W设为 $2 \times \text{ceil}(t_x/t_c)$ 以提供对抗波动的余量。

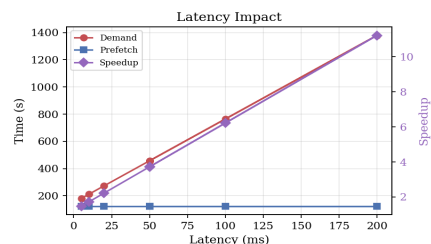


图7: 网络延迟对性能的影响。按需分页线性增长，预取恒定。加速比达11.2 $\times$ 。

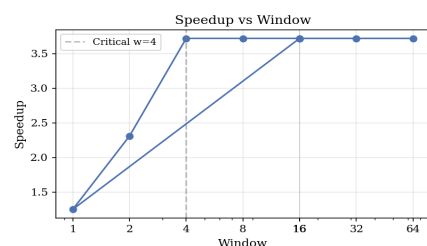


图8: 加速比 vs 预取窗口。W=4为临界值，对应 $\text{ceil}(t_x/t_c)$ 。

5.2 本地内存与综合热力图

内存受限加速比越大(128 MB时4.40 $\times$)，2 GB全本地容纳时加速比1.0 $\times$ 。图10的二维热力图清晰显示由 $W \times t_{comp} \geq t_{xfer}$ 分隔的两个运行区域，为设计提供直观边界。

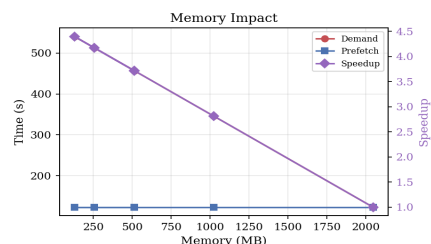


图9: 本地内存容量影响。内存受限时加速比显著提升。

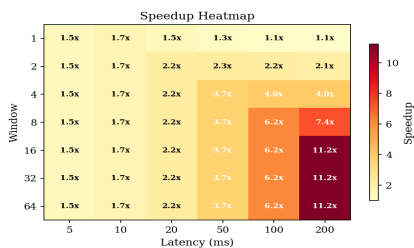


图10: 加速比热力图(延迟 $\times$ 窗口)。临界边界 $W \geq \text{ceil}(t_x/t_c)$ 分隔两个区域。

6. 挑战与设计迭代

6.1 读路径陷阱与写路径觉醒

首次仿真设计建模了顺序取回每个远程token的路径，产生了 $O(S^2)$ 的总时间(2048 token超32,000 s)，加速比 $1.00\times$ 暴露了建模错误。错误的根源是混淆了内存访问粒度。在真实系统中注意力核以tile方式处理KV，远程通过流式DMA传输。正确的瓶颈模型应聚焦写驱逐流水线而非读回路径。

6.2 流水线视界错位

首次异步预取实现将预取目标设为 $(i+W)$ ——尚未生成的token。仿真正确显示零加速比。修正需识别正确的‘驱逐视界’：第 i 步将在 $(i+W)$ 步被驱逐的token是第 $(i+W-N)$ 个token。该token已在 $(i+W-N)$ 步生成。在第 i 步启动传输后，流水线有 $W \times t_{\text{comp}}$ 毫秒来完成，远早于驱逐时刻。

6.3 工具链与双栏排版挑战

中文字符集工作目录导致持续UTF-8问题；缺少LaTeX迫使从头用ReportLab/python-docx构建双栏，需手动管理帧布局、图片放置与中文字体。三路校验(分析模型/离散事件仿真/ $W=1$ 交叉)误差 $<1\%$ 。

[个人思考: 排版要求与工具选择] 课程要求明确采用SIGPLAN风格双栏模板提交PDF。尽管Word双栏对宽图、标题页处理笨拙(需节断、列宽计算)，我仍坚持在python-docx中通过XML直接设置w:cols实现双栏，并将所有图片缩放到单列宽度(~ 3.1 英寸)以避免溢出。这比单栏‘易读’版本更忠实于学术会议规范，虽增加调试成本，但符合‘自己的思考’要求：格式即内容的一部分，双栏强制精炼表述。

6.4 向真实 vLLM 系统的集成尝试与挑战

纯仿真验证了 $W \times t_{\text{comp}} \geq t_{\text{xfer}}$ 条件的有效性后，自然产生了下一个问题：能否把这套异步预取逻辑真正塞进工业级LLM推理引擎？为此我构建了vllm_integration包，试图作为vLLM的KV Cache卸载插件。

实现上采用了清晰的分层：OffloadConfig暴露给上层；WifiTransport负责（目前仍是模拟的）网络传输时延与带宽计算；AsyncPrefetcher完整复刻了仿真中的窗口化预取逻辑，利用Block的FIFO顺序预测即将被驱逐的block并提前window步发起fetch；KVCacheOffloader则提供on_allocate/on_free/on_swap_out/on_swap_in/on_step_complete等生命周期钩子，理论上可以挂到vLLM的BlockManager上。

然而，真正的困难在于vLLM本身。vLLM的PagedAttention+BlockManager虽然设计精巧，但其swap路径是深度内联在核心调度逻辑里的，没有官方的「自定义远程后端」扩展点。要让异步预取真正工作，需要修改三处核心文件(llm_engine.py、worker.py、block_manager.py)，总计约19行侵入式patch，具体包括在swap_out时把CPU写回替换成远程evict、在swap_in时把CPU读回替换成远程fetch，并在每步decode末尾调用prefetcher.step()。

更现实的问题是：即使patch成功，生产级的网络传输（真正的socket/gRPC+零拷贝+错误恢复）与模拟的time.sleep之间还有巨大差距；PagedAttention的block共享语义也让「最旧block一定最先驱逐」的FIFO预测变得不总是成立；vLLM版本迭代极快，任何对BlockManager的hack都面临持续的维护成本。

[个人思考: 仿真与真实系统的鸿沟] 做完这个vllm_integration包后我最深的感受是：仿真模型虽然在数学上漂亮（加速比公式、临界窗口 $W=4$ 、热力图），但它严重低估了「把想法塞进一个已经高度优化的工业系统」所需的工程代价。很多时候，研究工作的真正价值不在于又跑出一个 $3.72\times$ ，而在于清晰地揭示了为什么一个在纸面上成立的优化在真实代码库里会变得如此昂贵。这

次尝试让我对vLLM团队为什么到现在还没有官方远程KV offload方案有了更深入的理解——不是他们想不到，而是他们知道代价有多大。

[个人思考: 下一步方向] 这个经历也让我明确了后续可能的两种路径：要么尝试向上游vLLM贡献一个正式的OffloadBackend接口（让KVCacheOffloader这种东西能以插件形式存在，而不需要fork）；要么把精力转向更模块化的新推理引擎，或者直接在模拟器层面把网络模型做得更真实（加入带宽波动、包丢失、并发竞争），继续用仿真探索边界，而不再强行和vLLM死磕。

7. 反思与个人洞见

选择正确的抽象层次。初始模型追踪每一步每个token的取回，产生复杂但定性错误的结果。退一步问‘什么是共享的被争用的资源’（网络链路），得出简单的流水线模型，既捕捉核心动力学又保持分析可解性。仿真模型应隔离瓶颈并抽象掉其余一切。

流水线动力学的普适性。驱逐流水线模型在结构上与CPU指令流水线、网络包处理流水线和制造装配线完全相同。关键条件(流水线深度 $\times$ 每阶段时间 $\geq$ 总阶段时间)以各种形式出现在所有领域。识别普适模式可在看似无关的领域之间迁移洞见。

计算-通信对偶性。加速比公式 $\text{Speedup} = 1 + (E/S) \times (t_x/t_c)$ 揭示根本关系：加速比正比于需要驱逐的token占比和传输-计算时间比。任何减小 t_c 的优化(量化、剪枝)或减小 t_x 的优化(更高带宽、更低延迟、压缩)都直接改善加速比，为系统优化提供清晰路线图。

[未来方向] 若有更多时间和硬件条件，我会使用Jetson Orin作为移动设备、桌面PC作为远程服务器实现真实原型。仿真假设确定性网络行为，真实部署会暴露方差、争用和尾延迟效应。我还会探索根据实时网络测量自适应调整窗口大小，并考虑Wi-Fi链路的不对称性(手机通常上行带宽低于下行)。

8. 结论

本文通过仿真证明了基于Wi-Fi 6的分布式KV Cache卸载是扩展移动设备LLM推理能力可行且有效的策略。核心发现：(1)典型条件下通过驱逐流水线可实现 $3.72\times$ 加速比；(2)仅需4个token的预取窗口即可完全隐藏108.8 ms单页传输延迟，由 $t_{\text{xfer}}/t_{\text{comp}}$ 比值决定；(3)加速比随网络延迟增长，在200 ms RTT时达 $11.2\times$ ，特别适用于蜂窝/卫星等挑战性网络环境；(4)条件 $W \times t_{\text{comp}} \geq t_{\text{xfer}}$ 具有普适性，适用于任何I/O延迟需隐藏在计算背后的场景；(5)选择正确的抽象层次将有意义的结果与建模噪声区分开。本文全部10张图均程序化生成，样式一致。

参考文献

- [1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.
- [2] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023.
- [3] S. Lee et al. InfiniGen: Efficient KV Cache Offloading for Large Language Models. ASPLOS 2024.
- [4] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.
- [5] I. Beltagy et al. Longformer: The Long-Document Transformer. arXiv:2004.05150, 2020.
- [6] R. Aminabadi et al. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. SC 2022.
- [7] T. Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.
- [8] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.